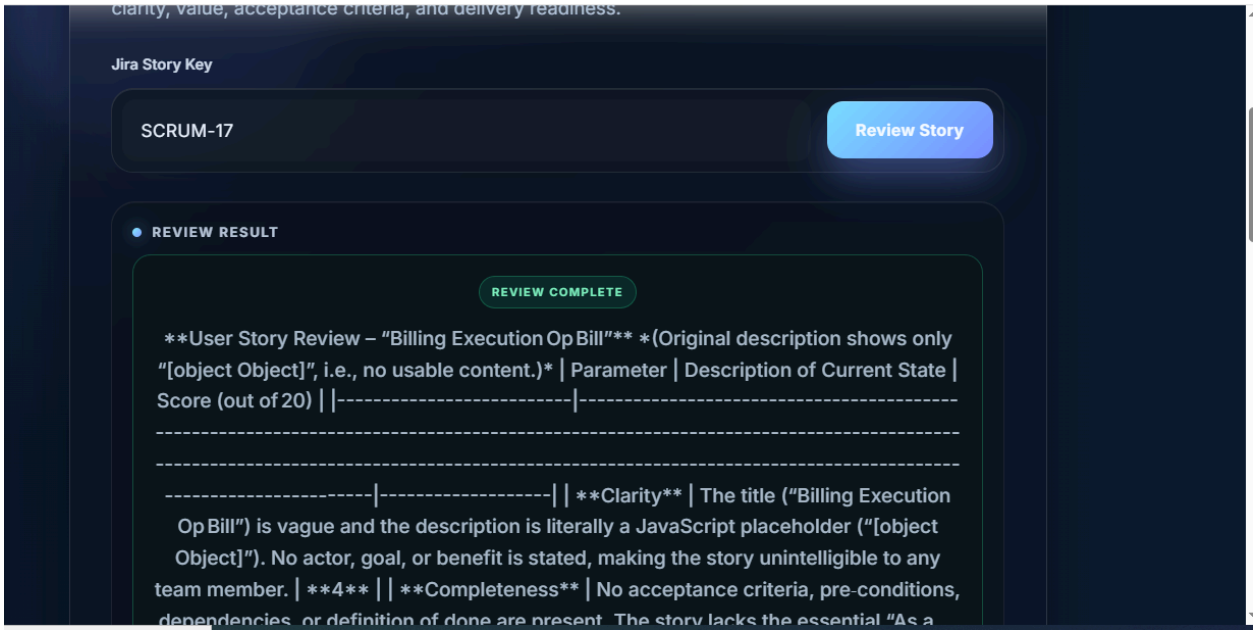
Phase2/Week 2/Day2 – Custom UI

Class Assignment: Custom UI through n8n for Jira user story



Home Assignment:

GitHub Code Reviewer Agent with Custom UI

Using the User Story Reviewer workflow and UI files as a reference, build a custom UI for the existing GitHub Code Reviewer Agent.

Update the n8n workflow by adding a Webhook node as the first node and a Respond to Webhook node as the final node. Create your own phase-wise UI development prompt, build the UI, and connect it to the n8n workflow.

Customize the UI based on your organization's branding, including:

- Organization name and logo

- Brand theme and colours

- Professional header and footer

- Suitable fonts, buttons, icons, and layout

- Responsive design for desktop, tablet, and mobile

- Loading, success, validation, and error messages

- Any other relevant organization-specific enhancements

Submission Requirements

Submit the following items in both the WhatsApp group and your GitHub repository:

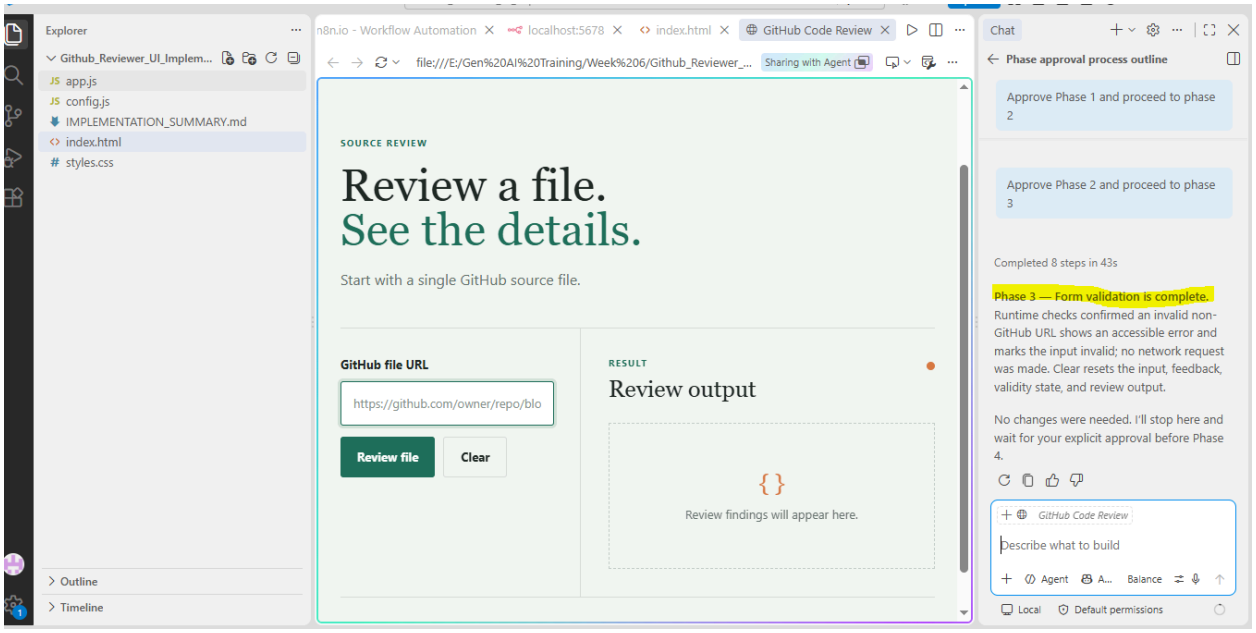
- Phase-wise implementation file

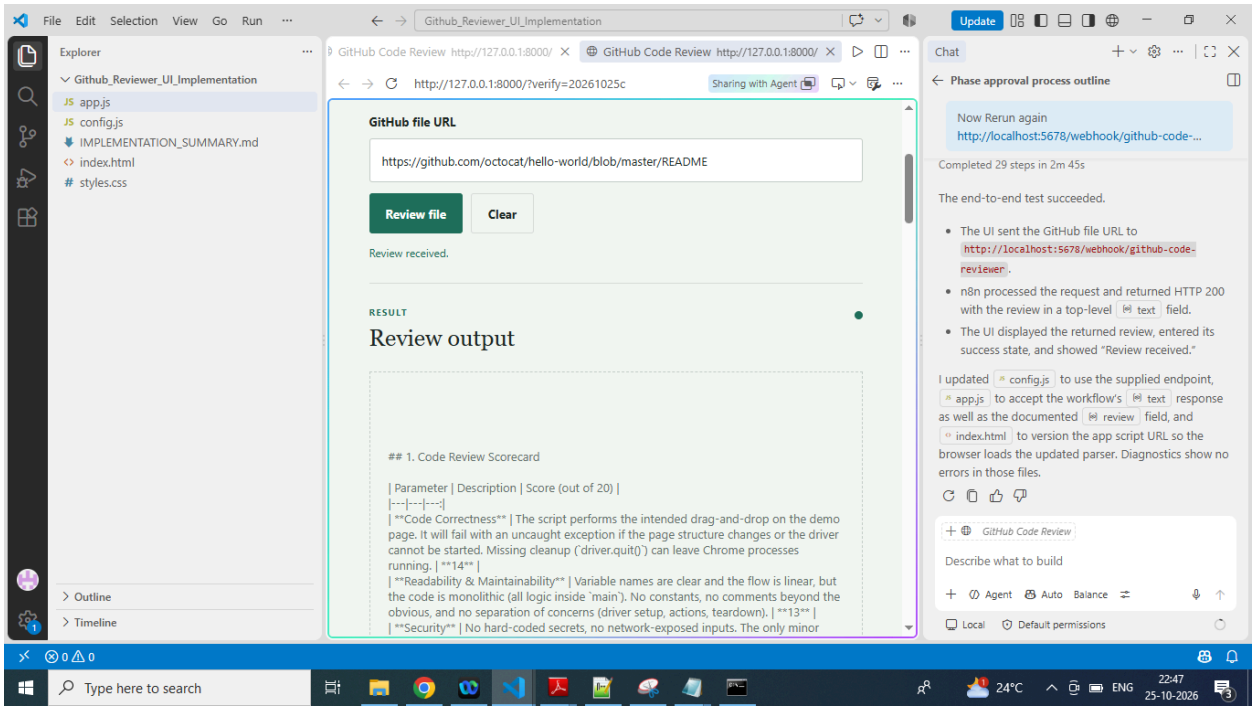
- Complete UI development prompt file

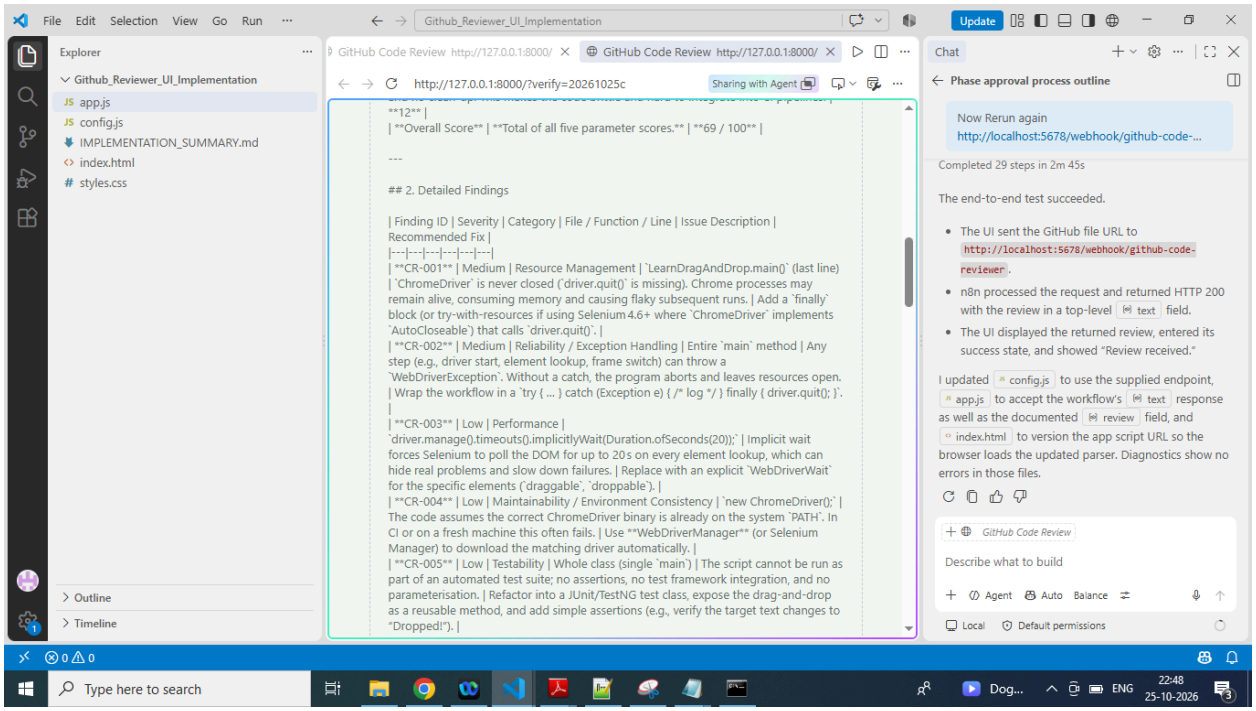
- n8n GitHub Code Reviewer workflow file

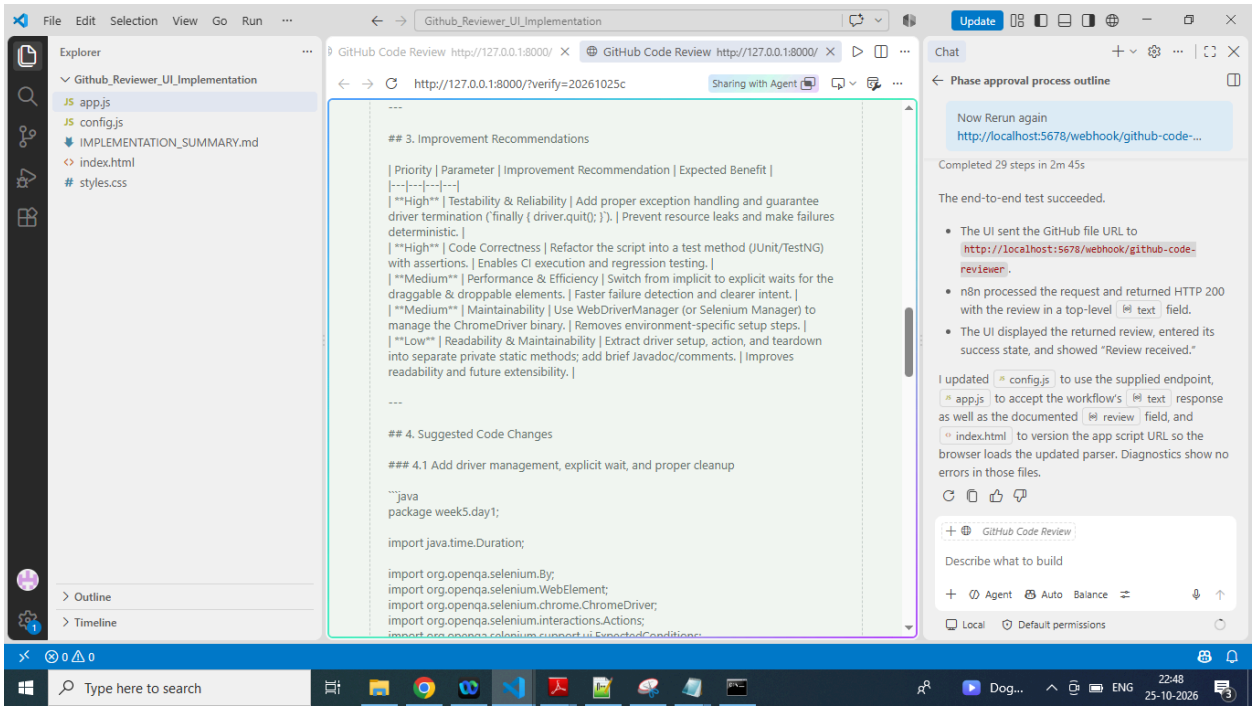
- Complete UI source-code files
- Screenshot of the complete n8n workflow
- Screenshots of the final UI and code-review results
- GitHub repository link
- Short demo video, if required

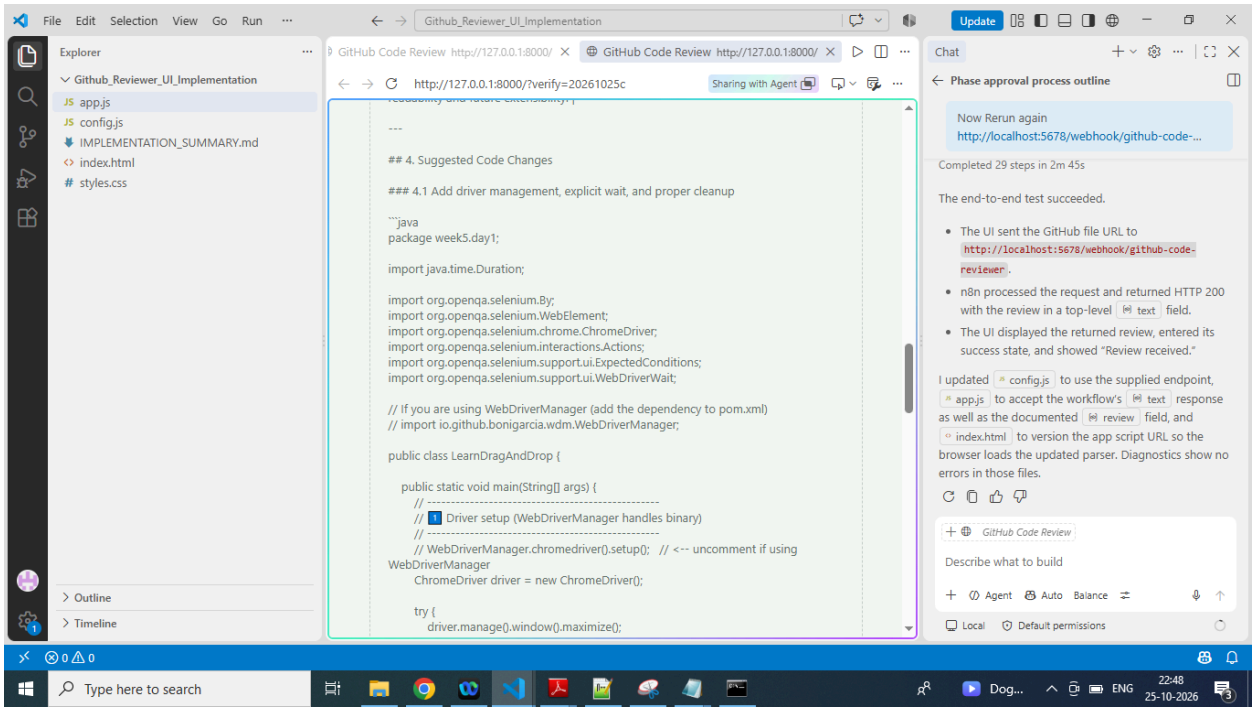
Ensure that the complete application is tested and working correctly before submission.

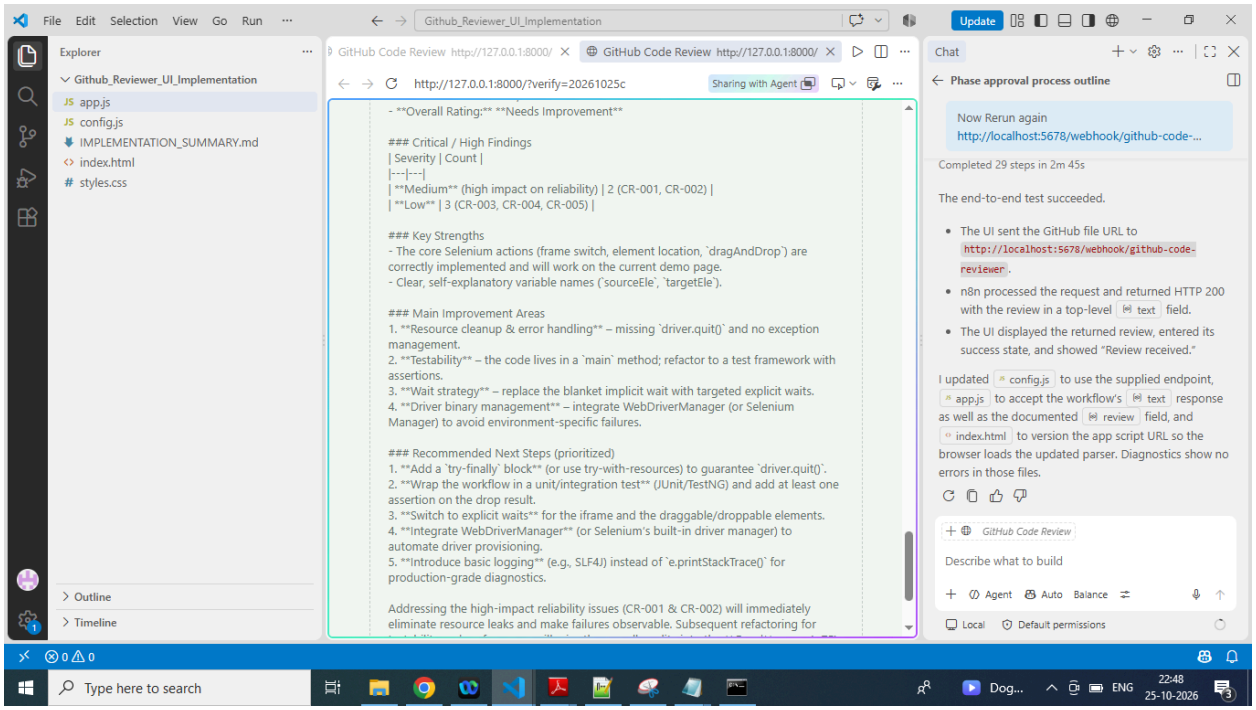



GitHub file URL
https://github.com/octocat/hello-world/blob/master/README
Review file
Clear
Review received.
RESULT
Review output
1. Code Review Scorecard
Parameter	Description	Score (out of 20)
Code Correctness	The script performs the intended drag-and-drop on the demo page. It will fail with an uncaught exception if the page structure changes or the driver cannot be started. Missing cleanup (`driver.quit()`) can leave Chrome processes running.	**14**
Readability & Maintainability	Variable names are clear and the flow is linear, but the code is monolithic (all logic inside `main`). No constants, no comments beyond the obvious, and no separation of concerns (driver setup, actions, teardown).	**13**
Security	No hard-coded secrets, no network-exposed inputs. The only minor
Phase approval process outline
Now Rerun again
http://localhost:5678/webhook/github-code-...
Completed 29 steps in 2m 45s
The end-to-end test succeeded.
The UI sent the GitHub file URL to http://localhost:5678/webhook/github-code-reviewer.
n8n processed the request and returned HTTP 200 with the review in a top-level text field.
The UI displayed the returned review, entered its success state, and showed "Review received."
I updated config.js to use the supplied endpoint, app.js to accept the workflow's text response as well as the documented review field, and index.html to version the app script URL so the browser loads the updated parser. Diagnostics show no errors in those files.
Describe what to build


12 |
| **Overall Score** | **Total of all five parameter scores.** | **69 / 100** |
2. Detailed Findings
Finding ID	Severity	Category	File / Function / Line	Issue Description	Recommended Fix
CR-001	Medium	Resource Management	`LearnDragAndDrop.main()` (last line)	`ChromeDriver` is never closed (`driver.quit()` is missing). Chrome processes may remain alive, consuming memory and causing flaky subsequent runs.	Add a `finally` block (or try-with-resources if using Selenium 4.6+ where `ChromeDriver` implements `AutoCloseable`) that calls `driver.quit()`.
CR-002	Medium	Reliability / Exception Handling	Entire `main` method	Any step (e.g., driver start, element lookup, frame switch) can throw a `WebDriverException`. Without a catch, the program aborts and leaves resources open.	Wrap the workflow in a `try { ... } catch (Exception e) { /* log */ } finally { driver.quit(); }`.
CR-003	Low	Performance	`driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(20));`	Implicit wait forces Selenium to poll the DOM for up to 20 s on every element lookup, which can hide real problems and slow down failures.	Replace with an explicit `WebDriverWait` for the specific elements (`draggable`, `droppable`).
CR-004	Low	Maintainability / Environment Consistency	`new ChromeDriver();`	The code assumes the correct ChromeDriver binary is already on the system `PATH`. In CI or on a fresh machine this often fails.	Use **WebDriverManager** (or Selenium Manager) to download the matching driver automatically.
CR-005	Low	Testability	Whole class (single `main`)	The script cannot be run as part of an automated test suite; no assertions, no test framework integration, and no parameterisation.	Refactor into a JUnit/TestNG test class, expose the drag-and-drop as a reusable method, and add simple assertions (e.g., verify the target text changes to "Dropped!").
Phase approval process outline
Now Rerun again
http://localhost:5678/webhook/github-code-...
Completed 29 steps in 2m 45s
The end-to-end test succeeded.
The UI sent the GitHub file URL to http://localhost:5678/webhook/github-code-reviewer.
n8n processed the request and returned HTTP 200 with the review in a top-level text field.
The UI displayed the returned review, entered its success state, and showed "Review received."
I updated config.js to use the supplied endpoint, app.js to accept the workflow's text response as well as the documented review field, and index.html to version the app script URL so the browser loads the updated parser. Diagnostics show no errors in those files.
Describe what to build

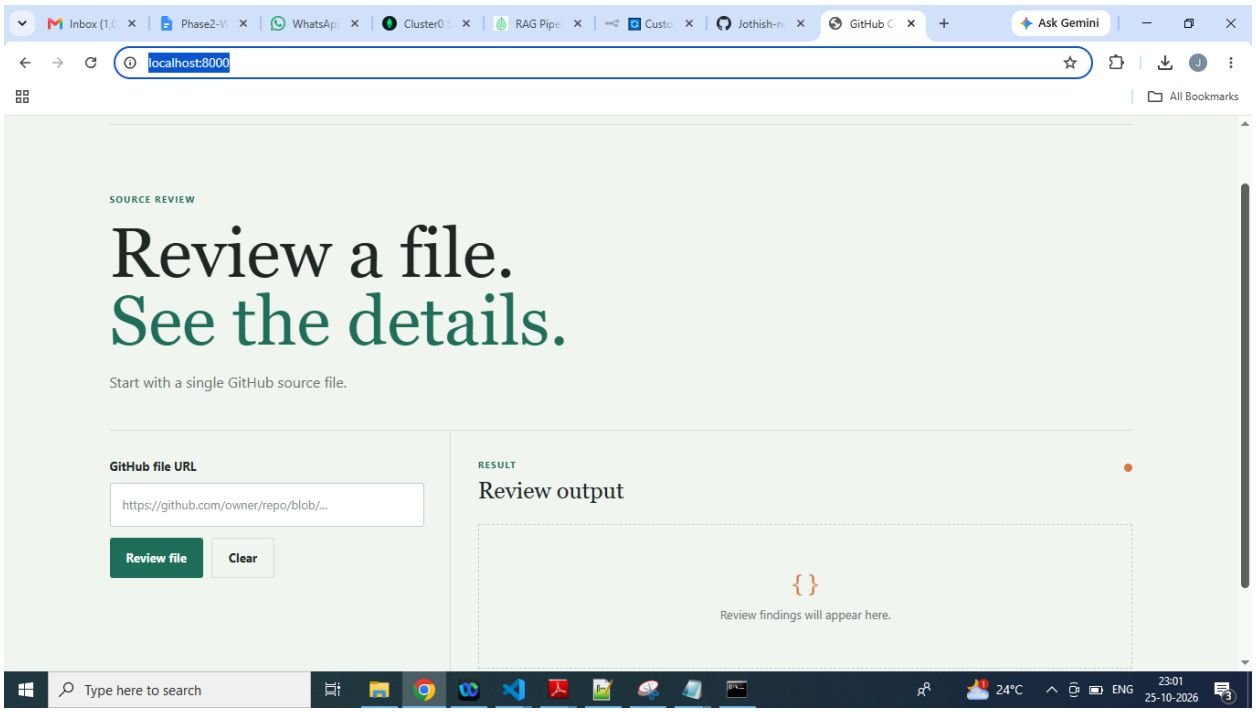

File Edit Selection View Go Run
Github_Reviewer_UI_Implementation
Update
Explorer
Github_Reviewer_UI_Implementation
app.js
config.js
IMPLEMENTATION_SUMMARY.md
index.html
styles.css
GitHub Code Review http://127.0.0.1:8000/
http://127.0.0.1:8000/?verify=20261025c
Sharing with Agent
3. Improvement Recommendations
Priority	Parameter	Improvement Recommendation	Expected Benefit
High	Testability & Reliability	Add proper exception handling and guarantee driver termination (`finally { driver.quit(); }`).	Prevent resource leaks and make failures deterministic.
High	Code Correctness	Refactor the script into a test method (JUnit/TestNG) with assertions.	Enables CI execution and regression testing.
Medium	Performance & Efficiency	Switch from implicit to explicit waits for the draggable & droppable elements.	Faster failure detection and clearer intent.
Medium	Maintainability	Use WebDriverManager (or Selenium Manager) to manage the ChromeDriver binary.	Removes environment-specific setup steps.
Low	Readability & Maintainability	Extract driver setup, action, and teardown into separate private static methods; add brief Javadoc/comments.	Improves readability and future extensibility.
4. Suggested Code Changes
4.1 Add driver management, explicit wait, and proper cleanup
```java
package week5.day1;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;
Outline
Timeline
Chat
Phase approval process outline
Now Rerun again
http://localhost:5678/webhook/github-code-...
Completed 29 steps in 2m 45s
The end-to-end test succeeded.
The UI sent the GitHub file URL to http://localhost:5678/webhook/github-code-reviewer.
n8n processed the request and returned HTTP 200 with the review in a top-level text field.
The UI displayed the returned review, entered its success state, and showed "Review received."
I updated config.js to use the supplied endpoint, app.js to accept the workflow's text response as well as the documented review field, and index.html to version the app script URL so the browser loads the updated parser. Diagnostics show no errors in those files.
GitHub Code Review
Describe what to build
Agent Auto Balance
Local Default permissions
Type here to search
Dog...
ENG
22:48
25-10-2026

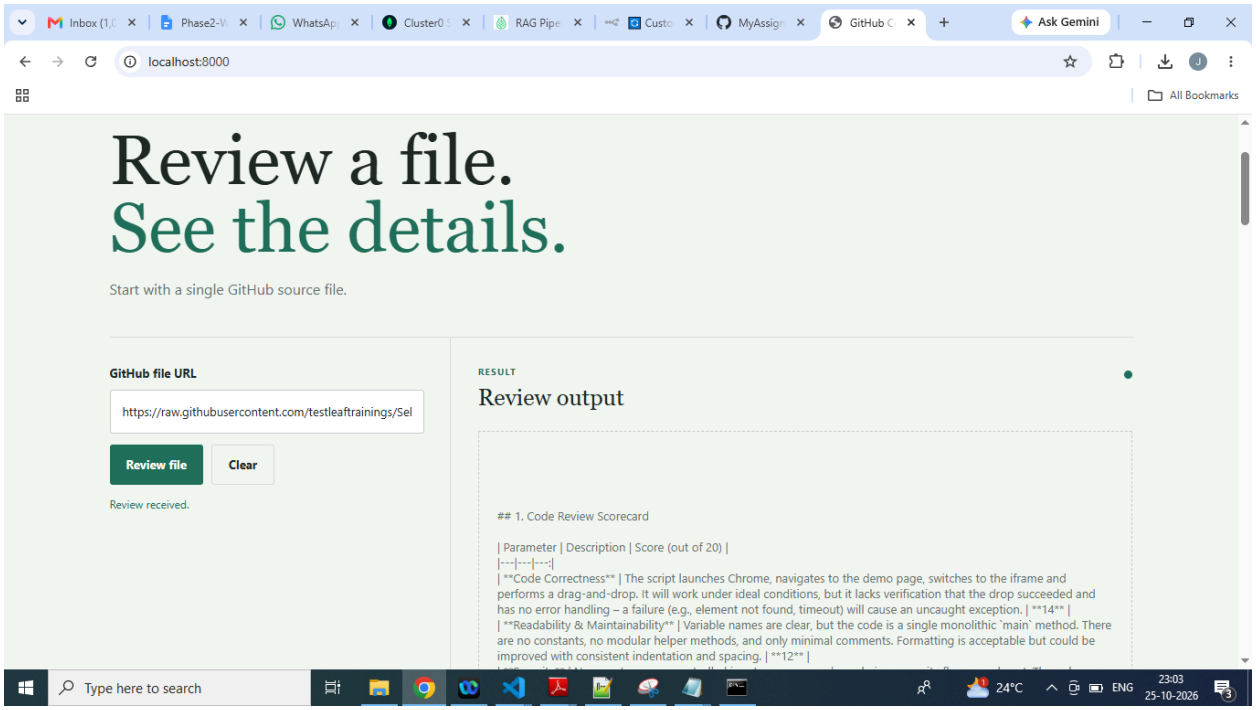

File Edit Selection View Go Run
Github_Reviewer_UI_Implementation
Update
Explorer
Github_Reviewer_UI_Implementation
app.js
config.js
IMPLEMENTATION_SUMMARY.md
index.html
styles.css
GitHub Code Review http://127.0.0.1:8000/
http://127.0.0.1:8000/?verify=20261025c
Sharing with Agent
## 4. Suggested Code Changes
### 4.1 Add driver management, explicit wait, and proper cleanup
```java
package week5.day1;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
// If you are using WebDriverManager (add the dependency to pom.xml)
// import io.github.bonigarcia.wdm.WebDriverManager;
public class LearnDragAndDrop {
public static void main(String[] args) {
// ----------------------------------------------------
// 1 Driver setup (WebDriverManager handles binary)
// ----------------------------------------------------
// WebDriverManager.chromedriver().setup(); // <-- uncomment if using WebDriverManager
ChromeDriver driver = new ChromeDriver();
try {
driver.manage().window().maximize();
Outline
Timeline
Chat
Phase approval process outline
Now Rerun again
http://localhost:5678/webhook/github-code-...
Completed 29 steps in 2m 45s
The end-to-end test succeeded.
The UI sent the GitHub file URL to http://localhost:5678/webhook/github-code-reviewer.
n8n processed the request and returned HTTP 200 with the review in a top-level text field.
The UI displayed the returned review, entered its success state, and showed "Review received."
I updated config.js to use the supplied endpoint, app.js to accept the workflow's text response as well as the documented review field, and index.html to version the app script URL so the browser loads the updated parser. Diagnostics show no errors in those files.
GitHub Code Review
Describe what to build
Agent Auto Balance
Local Default permissions
Type here to search
Dog...
ENG
22:48
25-10-2026

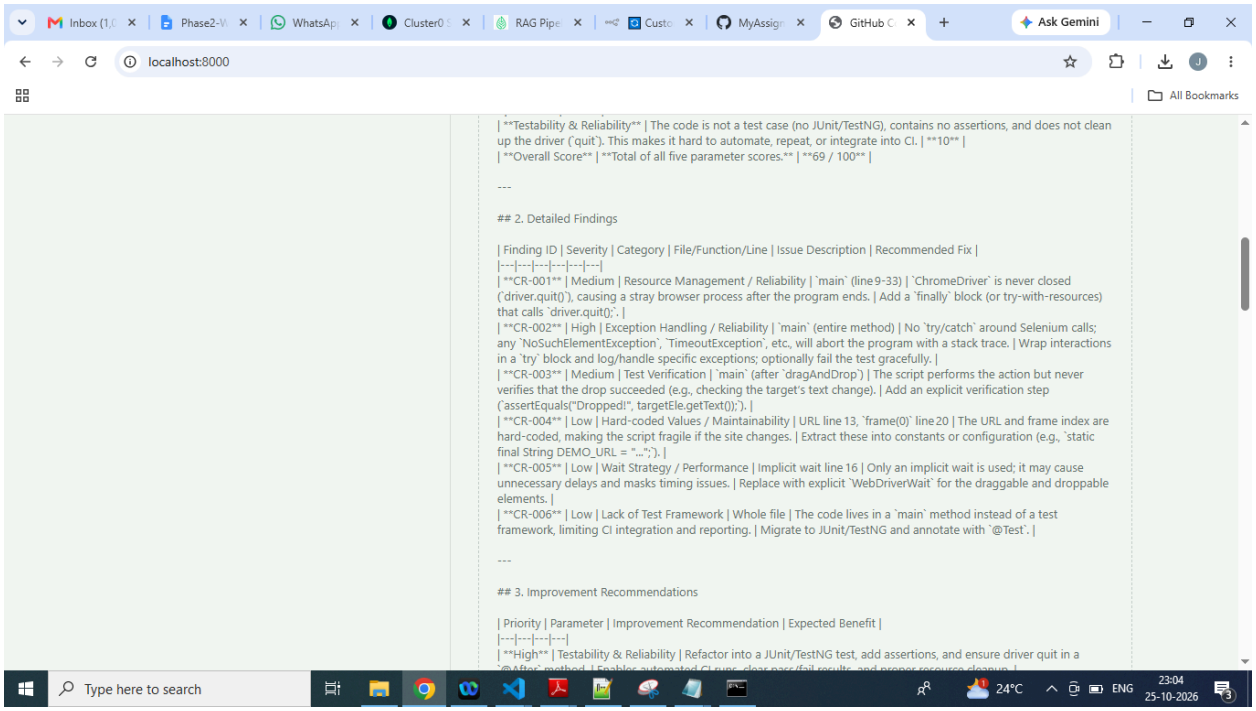


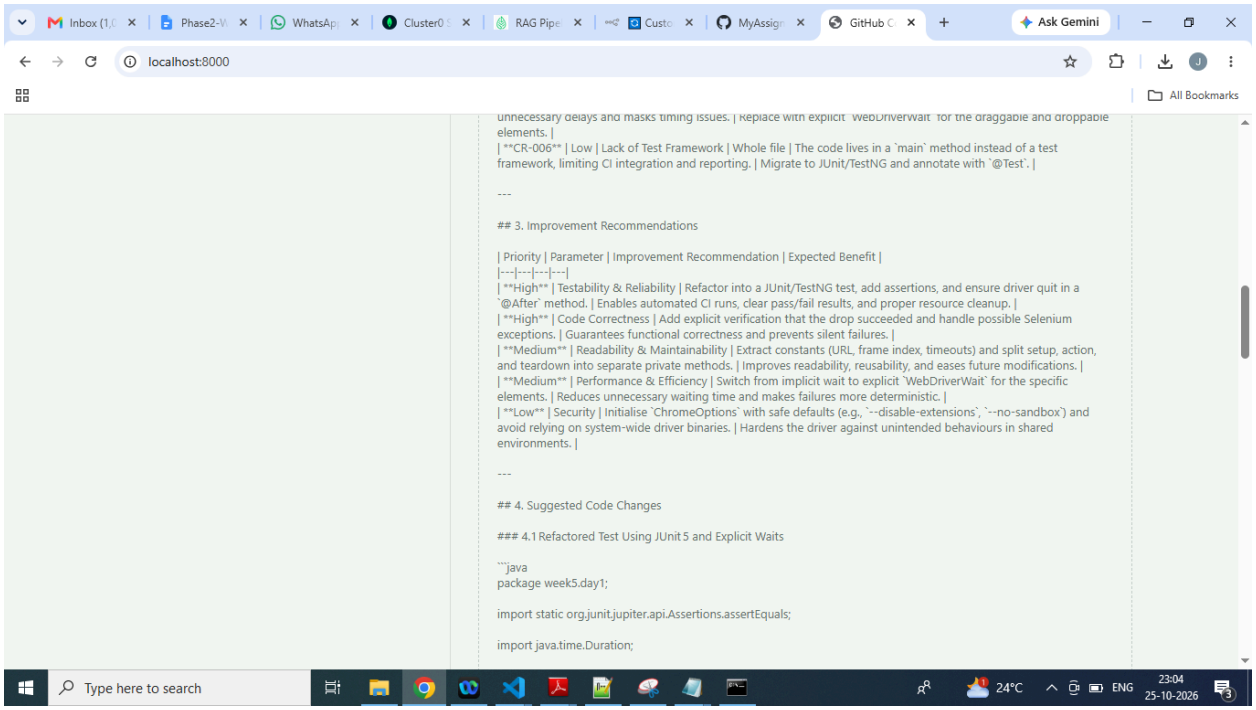

## Connecting to Local Host

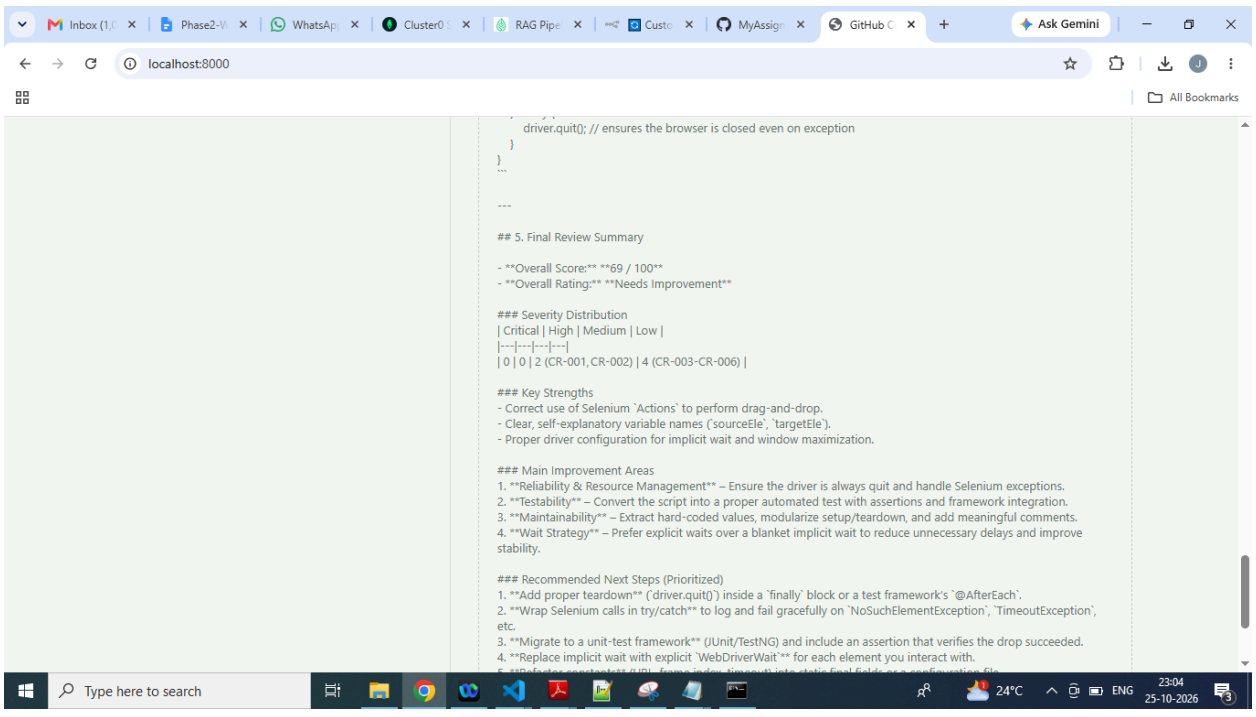
## [http://localhost:8000/](http://localhost:8000/)




localhost:8000
Review a file.
See the details.
Start with a single GitHub source file.
GitHub file URL
https://raw.githubusercontent.com/testleaftrainings/Sel
Review file
Clear
Review received.
RESULT
Review output
## 1. Code Review Scorecard
| Parameter | Description | Score (out of 20) |
|---|---|---|
| **Code Correctness** | The script launches Chrome, navigates to the demo page, switches to the iframe and performs a drag-and-drop. It will work under ideal conditions, but it lacks verification that the drop succeeded and has no error handling – a failure (e.g., element not found, timeout) will cause an uncaught exception. | **14** |
| **Readability & Maintainability** | Variable names are clear, but the code is a single monolithic `main` method. There are no constants, no modular helper methods, and only minimal comments. Formatting is acceptable but could be improved with consistent indentation and spacing. | **12** |
Type here to search
24°C
ENG
23:03
25-10-2026


localhost:8000
| **Testability & Reliability** | The code is not a test case (no JUnit/TestNG), contains no assertions, and does not clean up the driver (`quit`). This makes it hard to automate, repeat, or integrate into CI. | **10** |
| **Overall Score** | **Total of all five parameter scores.** | **69 / 100** |
---
## 2. Detailed Findings
| Finding ID | Severity | Category | File/Function/Line | Issue Description | Recommended Fix |
|---|---|---|---|---|---|
| **CR-001** | Medium | Resource Management / Reliability | `main` (line 9-33) | `ChromeDriver` is never closed (`driver.quit()`), causing a stray browser process after the program ends. | Add a `finally` block (or try-with-resources) that calls `driver.quit();`. |
| **CR-002** | High | Exception Handling / Reliability | `main` (entire method) | No `try/catch` around Selenium calls; any `NoSuchElementException`, `TimeoutException`, etc., will abort the program with a stack trace. | Wrap interactions in a `try` block and log/handle specific exceptions; optionally fail the test gracefully. |
| **CR-003** | Medium | Test Verification | `main` (after `dragAndDrop`) | The script performs the action but never verifies that the drop succeeded (e.g., checking the target's text change). | Add an explicit verification step (`assertEquals("Dropped!", targetEle.getText());`). |
| **CR-004** | Low | Hard-coded Values / Maintainability | URL line 13, `frame(0)` line 20 | The URL and frame index are hard-coded, making the script fragile if the site changes. | Extract these into constants or configuration (e.g., `static final String DEMO_URL = "...";`). |
| **CR-005** | Low | Wait Strategy / Performance | Implicit wait line 16 | Only an implicit wait is used; it may cause unnecessary delays and masks timing issues. | Replace with explicit `WebDriverWait` for the draggable and droppable elements. |
| **CR-006** | Low | Lack of Test Framework | Whole file | The code lives in a `main` method instead of a test framework, limiting CI integration and reporting. | Migrate to JUnit/TestNG and annotate with `@Test`. |
---
## 3. Improvement Recommendations
| Priority | Parameter | Improvement Recommendation | Expected Benefit |
|---|---|---|---|
| **High** | Testability & Reliability | Refactor into a JUnit/TestNG test, add assertions, and ensure driver quit in a
Type here to search
24°C
ENG
23:04
25-10-2026


localhost:8000
unnecessary delays and masks timing issues. | Replace with explicit `WebDriverWait` for the draggable and droppable elements. |
| **CR-006** | Low | Lack of Test Framework | Whole file | The code lives in a `main` method instead of a test framework, limiting CI integration and reporting. | Migrate to JUnit/TestNG and annotate with `@Test`. |
---
## 3. Improvement Recommendations
| Priority | Parameter | Improvement Recommendation | Expected Benefit |
|---|---|---|---|
| **High** | Testability & Reliability | Refactor into a JUnit/TestNG test, add assertions, and ensure driver quit in a `@After` method. | Enables automated CI runs, clear pass/fail results, and proper resource cleanup. |
| **High** | Code Correctness | Add explicit verification that the drop succeeded and handle possible Selenium exceptions. | Guarantees functional correctness and prevents silent failures. |
| **Medium** | Readability & Maintainability | Extract constants (URL, frame index, timeouts) and split setup, action, and teardown into separate private methods. | Improves readability, reusability, and eases future modifications. |
| **Medium** | Performance & Efficiency | Switch from implicit wait to explicit `WebDriverWait` for the specific elements. | Reduces unnecessary waiting time and makes failures more deterministic. |
| **Low** | Security | Initialise `ChromeOptions` with safe defaults (e.g., `--disable-extensions`, `--no-sandbox`) and avoid relying on system-wide driver binaries. | Hardens the driver against unintended behaviours in shared environments. |
---
## 4. Suggested Code Changes
### 4.1 Refactored Test Using JUnit 5 and Explicit Waits
```java
package week5.day1;
import static org.junit.jupiter.api.Assertions.assertEquals;
import java.time.Duration;
Type here to search
24°C
ENG
23:04
25-10-2026


localhost:8000
driver.quit(); // ensures the browser is closed even on exception
}
}
```
---
5. Final Review Summary
- **Overall Score:** **69 / 100**
- **Overall Rating:** **Needs Improvement**
Severity Distribution
Critical	High	Medium	Low
0	0	2 (CR-001, CR-002)	4 (CR-003-CR-006)
Key Strengths
- Correct use of Selenium `Actions` to perform drag-and-drop.
- Clear, self-explanatory variable names (`sourceEle`, `targetEle`).
- Proper driver configuration for implicit wait and window maximization.
Main Improvement Areas
1. **Reliability & Resource Management** – Ensure the driver is always quit and handle Selenium exceptions.
2. **Testability** – Convert the script into a proper automated test with assertions and framework integration.
3. **Maintainability** – Extract hard-coded values, modularize setup/teardown, and add meaningful comments.
4. **Wait Strategy** – Prefer explicit waits over a blanket implicit wait to reduce unnecessary delays and improve stability.
Recommended Next Steps (Prioritized)
1. **Add proper teardown** (`driver.quit()`) inside a `finally` block or a test framework's `@AfterEach`.
2. **Wrap Selenium calls in try/catch** to log and fail gracefully on `NoSuchElementException`, `TimeoutException`, etc.
3. **Migrate to a unit-test framework** (JUnit/TestNG) and include an assertion that verifies the drop succeeded.
4. **Replace implicit wait with explicit `WebDriverWait`** for each element you interact with.
Type here to search
24°C
ENG
23:04
25-10-2026